Spring JdbcTemplate, DataSource & HikariCP — Complete Notes

> This continues from the JDBC basics. Here we go deep into **how Spring wires everything together automatically** — from reading `application.properties` to a working `Connection` in your hand — and exactly what HikariCP is doing behind the scenes.

---

1. Big Picture — The Full Flow

```
application.properties/yml
        |
        ▼
Spring Boot Auto-Configuration
        |
        ▼
DataSource Bean Created  (HikariCP by default)
        |
        ▼
   HikariCP Connection Pool
   (pre-opens & holds real Connections)
        |
        ▼
  JdbcTemplate Bean Created
  (auto-wired with the DataSource)
        |
        ▼
Your Repository/DAO calls jdbcTemplate.query(...)
        |
        ▼
JdbcTemplate borrows a Connection from the pool
        |
        ▼
Creates PreparedStatement → executes SQL → gets ResultSet
        |
        ▼
```

```
Maps ResultSet rows to Java objects (RowMapper)
        |
        ▼
Connection is returned to the pool (NOT physically closed)
        |
        ▼
Result returned to your code
```

Keep this diagram in mind — every section below is zooming into one box of this picture.

---

2. Node: DataSource — the foundation

What it is: `javax.sql.DataSource` is an interface representing a **factory for database connections**. It's the standard, modern replacement for calling `DriverManager.getConnection()` directly.

Why Spring insists on `DataSource` instead of `DriverManager`:

Aspect	DriverManager	DataSource
Connection creation	New raw connection every call (slow — TCP handshake + auth each time)	Pulled from a **pool** of pre-created, ready connections (fast)
Configuration	Hardcoded in Java code	Externalized to `application.properties`, environment variables, JNDI
Pooling	None built-in	Delegates to a pooling library (HikariCP, DBCP2, C3P0)
Spring integration	Manual wiring	Auto-configured as a Spring Bean, injectable anywhere

Key method:

```
Connection getConnection() throws SQLException;
```

That's essentially the entire contract — anything that can hand you a `Connection` is a `DataSource`. The magic is in *how* it produces that connection (pooled vs. not).

---

3. Node: Connection Pooling — the core concept

Problem it solves: Opening a database connection is expensive — TCP handshake, authentication, session setup on the DB server. Doing this for every single query would make an application painfully slow under load.

Solution — a pool:

- At application startup, a fixed number of real connections are opened **once** and kept alive.
- When your code needs a connection, it **borrows** one from the pool (instant — no handshake).
- When done, the connection is **returned** to the pool (not physically closed) so another request can reuse it.
- If all connections are busy, new requests **wait** (up to a timeout) or the pool grows (up to a max size).

Analogy: Like a taxi stand instead of buying a new car for every trip — a fixed fleet of taxis (connections) waits; you take one, use it, and give it back for the next passenger.

Pool lifecycle terms:

Term	Meaning
Pool size	Number of connections kept ready
Minimum idle	Smallest number of idle (unused) connections kept alive
Maximum pool size	Hard cap on total connections (idle + in-use)
Connection timeout	How long a thread waits for a free connection before failing
Idle timeout	How long an unused connection sits before being closed/retired
Max lifetime	Maximum age of a connection before it's retired and replaced (avoids stale/broken connections)
Leak detection	Warns/logs if a connection is borrowed but never returned (a bug in your code)

---

4. Node: HikariCP — Spring Boot's default pool

What it is: HikariCP is a lightweight, extremely fast JDBC connection pooling library. Since **Spring Boot 2.0**, it is the **default** connection pool used automatically if it's on the classpath (which it is, by default, via `spring-boot-starter-jdbc` / `spring-boot-starter-data-jpa`).

Why Spring Boot chose HikariCP over alternatives (Apache DBCP2, C3P0, Tomcat pool):

- Very low latency and high throughput (benchmarks consistently show it's faster).
- Small, simple codebase — fewer moving parts, fewer bugs.

- Smart defaults — works well even with minimal configuration.

Core class: `com.zaxxer.hikari.HikariDataSource` — this is the actual `DataSource` implementation Spring registers as a bean.

Key configuration properties (`application.properties`):

```
spring.datasource.url=jdbc:mysql://localhost:3306/school_db
spring.datasource.username=root
spring.datasource.password=secret
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# HikariCP-specific tuning (all prefixed spring.datasource.hikari.*)
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=2
spring.datasource.hikari.idle-timeout=30000
spring.datasource.hikari.max-lifetime=1800000
spring.datasource.hikari.connection-timeout=20000
spring.datasource.hikari.pool-name=MyHikariPool
```

What each does:

Property	Meaning	Typical default
`maximum-pool-size`	Max connections in the pool (idle + active)	10
`minimum-idle`	Minimum idle connections HikariCP tries to maintain	same as max-pool-size
`connection-timeout`	Max ms a thread waits for a connection before throwing `SQLTransientConnectionException`	30000 (30s)
`idle-timeout`	Max ms a connection can sit idle before being retired (only if pool size > minimum-idle)	600000 (10 min)
`max-lifetime`	Max ms a connection is kept before being retired & replaced, even if in use	1800000 (30 min)
`pool-name`	Name shown in logs/JMX, useful when multiple pools exist	auto-generated

Why `max-lifetime` matters: Databases and network infrastructure (firewalls, load balancers) sometimes silently kill long-lived idle connections. HikariCP proactively retires connections before that happens, avoiding mysterious "connection is closed" errors in production.

---

5. Node: How Spring Boot Auto-Registers the DataSource Bean

This is the "magic" people often wonder about. Here's exactly what happens at startup:

Step-by-step:

1. Spring Boot sees `spring-boot-starter-jdbc` (or `-data-jpa`) on the classpath, which pulls in HikariCP as a transitive dependency.
2. `DataSourceAutoConfiguration` (a Spring Boot auto-configuration class) kicks in.
3. It reads `spring.datasource.*` properties into a `DataSourceProperties` object.
4. It checks the classpath for available pool implementations in a **preference order**: **HikariCP → Tomcat pool → Commons DBCP2** — whichever is found first is used.
5. Since HikariCP is present by default, Spring Boot builds a `HikariDataSource`, sets its properties (URL, username, password, pool settings) from `DataSourceProperties`, and registers it as a **Spring Bean** named `dataSource`.
6. This bean becomes available for `@Autowired`/constructor injection anywhere in your app.

Simplified view of what Spring does internally (conceptually equivalent to):

```
@Bean
public DataSource dataSource() {
    HikariConfig config = new HikariConfig();
    config.setJdbcUrl("jdbc:mysql://localhost:3306/school_db");
    config.setUsername("root");
    config.setPassword("secret");
    config.setMaximumPoolSize(10);
    return new HikariDataSource(config);
}
```

You never actually write this — Spring Boot's auto-configuration does it for you based on your properties file. You *can* override it manually with your own `@Bean` if you need custom logic (e.g., multiple databases).

Manual/explicit registration (if you want full control):

```
@Configuration
public class DataSourceConfig {

    @Bean
    public DataSource dataSource() {
```

```
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:postgresql://localhost:5432/mydb");
        config.setUsername("postgres");
        config.setPassword("secret");
        config.setMaximumPoolSize(15);
        config.setPoolName("CustomPool");
        return new HikariDataSource(config);
    }
}
```

---

6. Node: How Spring Registers `JdbcTemplate`

Auto-configuration class: `JdbcTemplateAutoConfiguration`.

What happens:

1. Spring Boot detects that a `DataSource` bean exists (created in the step above).
2. `JdbcTemplateAutoConfiguration` automatically creates a `JdbcTemplate` bean, **injecting that** `DataSource` into its constructor.
3. It also registers a `NamedParameterJdbcTemplate` bean the same way.
4. Both beans are now available for injection into any `@Repository`, `@Service`, or `@Component`.

Conceptually equivalent to:

```
@Bean
public JdbcTemplate jdbcTemplate(DataSource dataSource) {
    return new JdbcTemplate(dataSource);
}
```

Using it in your code — no manual wiring needed:

```
@Repository
public class StudentRepository {

    private final JdbcTemplate jdbcTemplate;

    // Spring injects the auto-configured JdbcTemplate bean here
    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
```

```
    }
}
```

Condition for auto-configuration to trigger: Spring Boot only creates `JdbcTemplate` automatically if:

- `spring-boot-starter-jdbc` is on the classpath, **and**
- A `DataSource` bean is present, **and**
- You haven't already defined your own `JdbcTemplate` bean (Spring Boot backs off if you did — `@ConditionalOnMissingBean`).

---

7. Node: Inside `JdbcTemplate` — What It Actually Does Per Call

When you call, say:

```
jdbcTemplate.query("SELECT * FROM students WHERE age > ?", new StudentRow
```

Here is the **exact internal flow**:

1. **Borrow a connection** — `JdbcTemplate` calls `DataSourceUtils.getConnection(dataSource)`, which asks HikariCP for a pooled `Connection`.
    - HikariCP hands over an already-open, ready-to-use physical connection (no new TCP handshake).
2. **Create a `PreparedStatement`** — `connection.prepareStatement(sql)`, then binds parameters (`18` → the `?`).
3. **Execute** — calls `preparedStatement.executeQuery()`, which sends the SQL to the DB and gets back a `ResultSet`.
4. **Map results** — iterates the `ResultSet`, calling your `RowMapper.mapRow(rs, rowNum)` for every row, collecting the results into a `List<Student>`.
5. **Clean up (in a `finally` block, guaranteed even on error):**
    - `ResultSet.close()`
    - `PreparedStatement.close()`
    - **Connection is NOT physically closed** — `DataSourceUtils.releaseConnection()` returns it to the HikariCP pool for reuse.

6. **Exception translation** — if any `SQLException` occurred anywhere in this flow, `JdbcTemplate` catches it and converts it into the appropriate Spring `DataAccessException` subclass using `SQLExceptionTranslator` (so your code deals with one consistent exception hierarchy, not vendor-specific SQL error codes).
7. **Return the result** to your calling code — a plain Java `List<Student>`, no JDBC types leaked out.

This is the entire value proposition of `JdbcTemplate`: it performs steps 1–6 for you on *every single call*, so your code only ever writes step 7's worth of logic (the SQL + how to map a row).

---

8. Node: Connection Borrow/Return Cycle in Detail

```
Idle Connections in Pool: [C1] [C2] [C3] [C4] [C5]
                                  |
        jdbcTemplate.query() calls DataSourceUtils.getConnection()
                                  |
                                  ▼
                      HikariCP hands out C3
                                  |
Idle Connections in Pool: [C1] [C2] [C4] [C5]     Active: [C3]
                                  |
               PreparedStatement created on C3, SQL executed
                                  |
                        ResultSet processed
                                  |
        finally block → DataSourceUtils.releaseConnection(C3)
                                  |
                                  ▼
                  C3 returned to pool (physical socket stays open)
                                  |
Idle Connections in Pool: [C1] [C2] [C3] [C4] [C5]
```

Important nuance: Calling `connection.close()` on a pooled connection does **not** actually close the TCP socket — HikariCP's wrapper intercepts `close()` and instead returns the connection to the pool. This is what makes pooling transparent to code that's used to plain JDBC's `close()` semantics.

---

9. Node: Transactions & the Pool Working Together

When you use `@Transactional`:

```
@Service
public class StudentService {
    @Transactional
    public void transferMarks(int fromId, int toId, int marks) {
        studentRepository.deduct(fromId, marks);
        studentRepository.add(toId, marks);
    }
}
```

What happens:

1. Spring's `DataSourceTransactionManager` (auto-configured alongside `DataSource`) intercepts the method call via a proxy.
2. It borrows **one** connection from the HikariCP pool and **binds it to the current thread** (via `TransactionSynchronizationManager`).
3. It sets `connection.setAutoCommit(false)`.
4. **Every** `JdbcTemplate` call inside this method (both `deduct` and `add`) automatically reuses that **same bound connection** instead of borrowing a new one — this is essential, otherwise each call would be its own separate mini-transaction.
5. If the method completes normally → `connection.commit()`.
6. If a `RuntimeException` is thrown → `connection.rollback()`.
7. Either way, the connection is then released back to the pool.

This thread-bound connection mechanism is why `@Transactional` "just works" without you ever touching a `Connection` object.

---

10. Node: HikariCP Functionalities Summary

Functionality	What it does
Pooling	Reuses a fixed set of physical connections instead of creating new ones per request
Fast connection acquisition	Uses lock-free/low-overhead data structures internally for speed
Connection validation	Tests connections before handing them out (via JDBC4 `isValid()` or a test query) to avoid giving out dead connections
Leak detection	`leak-detection-threshold` logs a warning if a connection is held too long without being returned

Functionality	What it does
Automatic retirement	Retires connections past `max-lifetime` or `idle-timeout` and replaces them transparently
Metrics/JMX support	Exposes pool stats (active, idle, waiting threads) for monitoring (works well with Micrometer/Actuator)
Fail-fast startup	Can be configured to fail application startup immediately if it can't establish initial connections

Monitoring via Spring Boot Actuator (if enabled):

```
GET /actuator/metrics/hikaricp.connections.active
GET /actuator/metrics/hikaricp.connections.idle
GET /actuator/metrics/hikaricp.connections.pending
```

---

11. Node: Multiple DataSources (advanced but common in real apps)

When an app needs to talk to **two databases**, auto-configuration only handles one automatically — you define beans explicitly and mark one `@Primary`:

```
@Configuration
public class MultiDataSourceConfig {

    @Primary
    @Bean(name = "primaryDataSource")
    @ConfigurationProperties(prefix = "spring.datasource.primary")
    public DataSource primaryDataSource() {
        return DataSourceBuilder.create().type(HikariDataSource.class).bu
    }

    @Bean(name = "secondaryDataSource")
    @ConfigurationProperties(prefix = "spring.datasource.secondary")
    public DataSource secondaryDataSource() {
        return DataSourceBuilder.create().type(HikariDataSource.class).bu
    }

    @Bean
    public JdbcTemplate primaryJdbcTemplate(@Qualifier("primaryDataSource
        return new JdbcTemplate(ds);
    }
```

```
    @Bean
    public JdbcTemplate secondaryJdbcTemplate(@Qualifier("secondaryDataSc
        return new JdbcTemplate(ds);
    }
}
```

```
spring.datasource.primary.jdbc-url=jdbc:mysql://localhost:3306/db1
spring.datasource.primary.username=root
spring.datasource.primary.password=secret

spring.datasource.secondary.jdbc-url=jdbc:postgresql://localhost:5432/db2
spring.datasource.secondary.username=postgres
spring.datasource.secondary.password=secret
```

Each `DataSource` gets its **own independent HikariCP pool**.

---

12. Node: Complete End-to-End Flow (start to finish, one query)

```
1. App starts
   → Spring Boot reads application.properties
   → DataSourceAutoConfiguration builds a HikariDataSource bean
     (pool pre-warms: opens `minimum-idle` real connections to the DB)
   → JdbcTemplateAutoConfiguration builds a JdbcTemplate bean wrapping
that DataSource
   → DataSourceTransactionManager bean is also registered

2. HTTP request hits a @RestController
   → Controller calls a @Service method annotated @Transactional
   → Spring's transaction proxy intercepts the call
     → borrows 1 connection from HikariCP pool, binds to current thread
     → sets autoCommit = false

3. @Service calls @Repository method
   → Repository calls jdbcTemplate.query(sql, rowMapper, params)
     → JdbcTemplate detects a thread-bound connection exists (from step 2)
and reuses it
        (or borrows a fresh one from the pool if no transaction is active)
```

```
     → creates PreparedStatement, binds params
     → executes SQL against the real database
     → gets ResultSet back
     → RowMapper converts each row into a Java object
     → PreparedStatement & ResultSet are closed
     → Connection is NOT closed — released back toward the pool
       (but if inside a transaction, stays bound to thread until the
transaction ends)

4. @Service method finishes normally
   → transaction proxy calls connection.commit()
   → connection is unbound from thread and returned to the HikariCP pool
     (physical socket stays open, just marked idle again)

5. Result (List<Student>, etc.) flows back up:
   Repository → Service → Controller → HTTP response to client

6. If any RuntimeException occurred anywhere in step 2-3:
   → transaction proxy calls connection.rollback() instead of commit()
   → connection still returned to the pool afterward (pool doesn't care
about your business errors)
```

---

13. Quick Reference — Bean Wiring Chain

```
application.properties (spring.datasource.*)
        |
        ▼
DataSourceProperties          (Spring Boot reads properties into this)
        |
        ▼
DataSourceAutoConfiguration  →  HikariDataSource bean ("dataSource")
        |
        ▼
JdbcTemplateAutoConfiguration →  JdbcTemplate bean (wraps "dataSource")
                              →  NamedParameterJdbcTemplate bean
        |
        ▼
DataSourceTransactionManager bean (also wraps "dataSource", enables
@Transactional)
        |
```

▼

```
Your @Repository / @Service classes — just @Autowired or constructor-inject
JdbcTemplate and start calling .query()/.update(), everything else is automatic
```

---

14. Common Pitfalls & Best Practices

Pitfall	Why it's a problem	Fix
Setting `maximum-pool-size` too high	More connections ≠ more speed; DB server has its own connection limits, and contention/context-switching can slow things down	Start around 10, tune based on actual load testing (HikariCP wiki has a sizing formula)
Not setting `max-lifetime` below the DB/network's own idle-kill timeout	You get random "connection closed" errors in production	Set `max-lifetime` a few minutes less than your DB or firewall's timeout
Long-running code inside `@Transactional` methods (e.g., calling external APIs)	Holds a pooled connection the whole time, starving the pool under load	Keep transactional methods short — DB work only
Manually calling `DriverManager.getConnection()` inside a Spring app	Bypasses the pool entirely — defeats the purpose	Always inject and use `DataSource` / `JdbcTemplate`
Forgetting `@Transactional` is proxy-based	Calling a `@Transactional` method from *within the same class* (self-invocation) bypasses the proxy — no transaction applied	Call transactional methods from a different bean, or restructure

---

15. Mental Model to Remember

```
application.properties  →  raw config values
DataSourceProperties    →  typed Java object holding those values
HikariDataSource        →  the real DataSource; owns and manages the connection pool
JdbcTemplate            →  wraps DataSource; automates borrow → execute → map → release
DataSourceTransactionManager → wraps DataSource; automates commit/rollback + thread-bound connections
```

```
Your Repository code   →  only ever writes: SQL + RowMapper — nothing
else
```

One-line summary: Spring Boot reads your properties, builds a HikariCP-backed `DataSource`, wraps it in a `JdbcTemplate` (and a `TransactionManager`), and from then on every `.query()` / `.update()` call transparently borrows a warm connection from the pool, runs your SQL safely via `PreparedStatement`, maps the result, and returns the connection — all without you ever touching `DriverManager`, `Connection.close()`, or raw `SQLException` handling.